TC Video Upload Service — API Documentation

Version 1.0 · Deployed on server **p010** (NayaSoft) · Node.js + Express, running in Docker.

1. Overview

A private microservice for uploading exam-centre videos and storing them, **raw and uncompressed**, in a folder named after the **TC code**. It accepts single videos, multiple videos, or `.zip` archives (from which only the video files are extracted).

- **Storage location on server:** `/localstorage/videos/<TC_CODE>/` (21 TB RAID disk)
- **Runs as:** Docker container `tc-upload` (auto-restarts, survives reboot)
- **Designed for:** 7–10 people uploading simultaneously, up to ~15 TB total
- **Integrity:** stored files are byte-identical to the source (verified by checksum)

2. Base URL & Access

The API is **private** — it is **not** exposed on the public internet (only SSH ports are port-forwarded on the public IP). Reach it over the local/private network:

From	Base URL
A frontend hosted on the same server (p010)	`http://127.0.0.1:8080` or `http://100.81.36.13:8080`
Another machine on the NayaSoft private network	`http://100.81.36.13:8080`

All endpoints return JSON. **CORS is enabled** (`Access-Control-Allow-Origin: *`), so a browser frontend served from any origin on the server can call it directly.

3. Endpoints

3.1 `POST /upload` — upload videos

Upload one or more videos and/or zip archives for a TC code.

Content-Type: `multipart/form-data`

Request fields

Field	Type	Required	Notes
`tcCode`	text	Yes	Folder name. Allowed: letters, digits, `-`, `_` (1–64 chars).
(any)	file	Yes	One or more files. **Field name is ignored** — send under any name(s). Each file is a video or a `.zip`.

Behavior

- Video files are saved as-is into `/localstorage/videos/<tcCode>/`.
- `.zip` files are extracted; **only video files inside are kept**, everything else is skipped.
- Duplicate filenames are auto-suffixed (`clip.mp4` → `clip_1.mp4`, `clip_2.mp4`).
- No re-encoding or compression — bytes are preserved.

Success response `200`

```
{
 "ok": true,
 "tcCode": "2403103",
 "directory": "/data/2403103",
 "saved_count": 2,
 "skipped_count": 1,
 "saved": [
   { "name": "DSC_0707.MOV", "size_bytes": 2390596, "source": "direct" },
   { "name": "DSC_0701.MOV", "size_bytes": 9329041, "source": "videos.zip" }
 ],
 "skipped": [
   { "file": "notes.txt", "reason": "not a video (inside zip)" }
 ]
}
```

`directory` shows the in-container path `/data/...`; on the host it is `/localstorage/videos/...`.

Error responses

Code	Meaning
`400`	Missing/invalid `tcCode`, or no files, or a malformed request
`507`	Insufficient storage — free space below the safety threshold (default 25 GB)
`500`	Unexpected server error

3.2 `GET /tc/:tcCode` — list stored videos

Returns the videos already stored for a TC code.

```
{ "tcCode": "2403103", "count": 2, "directory": "/data/2403103",
 "files": [ { "name": "DSC_0707.MOV", "size_bytes": 2390596, "modified": "2026-07-09T04:58:51.737Z" } ] }
```

3.3 `GET /stats` — storage/monitoring

```
{ "storage": "/data", "tc_codes": 12,
 "disk": { "total_gb": 22951, "used_gb": 1220.3, "free_gb": 21730.7, "used_pct": 5 },
 "min_free_gb": 26.8 }
```

3.4 `GET /health` — liveness

```
{ "ok": true, "time": "2026-07-09T05:01:24.709Z" }
```

3.5 `GET /` — service info

Returns the service description, storage path, endpoint list, and accepted video types.

4. Integration Examples

4.1 cURL

```
BASE=http://100.81.36.13:8080

# single video
curl -F tcCode=2403103 -F files=@DSC_0707.MOV $BASE/upload

# multiple videos (any field names)
curl -F tcCode=2403103 -F a=@v1.mp4 -F b=@v2.mov -F c=@v3.mkv $BASE/upload

# a zip bundle of many videos
curl -F tcCode=2403103 -F archive=@all_videos.zip $BASE/upload

# list / stats
curl $BASE/tc/2403103
curl $BASE/stats
```

4.2 Browser frontend (JavaScript `fetch`)

```
<input id="tc" placeholder="TC code">
<input id="files" type="file" multiple accept="video/*,.zip">
<button onclick="send()">Upload</button>
<script>
async function send() {
 const fd = new FormData();
 fd.append('tcCode', document.getElementById('tc').value);
 for (const f of document.getElementById('files').files) fd.append('files', f);
 const res = await fetch('http://100.81.36.13:8080/upload', { method: 'POST', body: fd });
 console.log(await res.json());
}
</script>
```

For a progress bar on large files, use `XMLHttpRequest` and listen to `upload.onprogress`.

4.3 Python

```
import requests
files = [('files', open('DSC_0707.MOV','rb')), ('files', open('DSC_0693.MOV','rb'))]
r = requests.post('http://100.81.36.13:8080/upload',
                  data={'tcCode': '2403103'}, files=files)
print(r.json())
```

5. Upload Limitations

#	Limitation	Detail
1	**Max file size**	64 GB per individual file.

2	**Max files per request**	5000 files in a single `/upload` call.
3	**Accepted video types**	`.mov` `.mp4` `.avi` `.mkv` `.m4v` `.wmv` `.flv` `.mpeg` `.mpg` `.webm` `.mts` `.m2ts` `.ts` `.3gp` `.3g2` `.vob` `.ogv` `.mxf` `.dv`. Any other file is skipped.
4	**Archive format**	Only `.zip`. `.rar`, `.7z`, `.tar`, `.tar.gz` are **not** supported.
5	**Detection is by file extension**, not content. A non-video renamed to `.mp4` is still stored; a real video with a wrong/missing extension is skipped.	

6	**Zip structure is flattened**	Videos inside sub-folders of a zip are placed directly in the TC folder (by filename).
7	**`tcCode` format**	Letters, digits, `-`, `_` only, 1–64 chars. Spaces, slashes, or `..` are rejected (prevents path traversal).
8	**Filename sanitisation**	Characters outside `A–Z` `a–z` `0–9` `.` `_` `-` are replaced with `_`.
9	**Duplicates are never overwritten**	Same name → auto-suffixed `_1`, `_2`, …

10	**No resumable/chunked upload**	If a connection drops mid-upload that file must be re-sent from the start. The client connection must stay open for the whole transfer.
11	**No authentication**	Anyone able to reach the port on the private network can upload. Security relies on network isolation.
12	**No per-user quota / rate limit**	All users share the same disk; there is no per-person cap.

13	**Plain HTTP (no TLS)**	Fine on a trusted private network; add TLS/nginx if traffic crosses untrusted links.
14	**Disk-full guard**	New uploads are refused with HTTP `507` when free space falls below 25 GB (configurab via `MIN_FREE_`
15	**Stored file ownership**	Files are owned by `root` (written by the container) and are world-reada — readable by other processes, but only removable via the API or Docker.

16	**Transient space for zips**	Extracting a zip briefly needs extra space (~the size of its videos) until the archive is cleaned up. Direct (non-zip) uploads need no extra space.

6. Concurrency & Scale (7–10 users, up to 15 TB)

The service is built and **verified** to handle the stated load:

- **Concurrent uploaders:** uploads stream directly to disk (never buffered in RAM), so 10 simultaneous multi-GB uploads do not exhaust memory. Each request uses an isolated temporary folder, so users never collide.
- **Race-safe file naming:** two people uploading a file with the **same name to the same TC code at the same instant** each get a distinct file — verified with 10 simultaneous identical uploads (result: 10 files, none lost, none overwritten).
- **Efficient storage:** direct videos are *moved* (not copied) into place on the same disk, so no extra space or copy time is incurred.
- **Capacity:** the `/localstorage` disk has **~21.7 TB free**; 15 TB fits with ~6 TB headroom. `/stats` exposes live usage and the 25 GB guard prevents a full-disk failure.
- **Throughput** is bounded by network and disk, not the service. For very high parallelism (dozens of users) the container can be scaled to multiple Node workers behind nginx.

7. Operations

```
# on the server (p010)
docker logs -f tc-upload           # live logs
docker restart tc-upload           # restart
docker stats tc-upload             # live CPU/mem
curl http://127.0.0.1:8080/stats   # disk usage

# update after editing code in ~/tc-upload-service
cd ~/tc-upload-service && docker build -t tc-upload:latest . \
 && docker rm -f tc-upload \
 && docker run -d --name tc-upload --restart unless-stopped \
      -p 8080:8080 -v /localstorage/videos:/data tc-upload:latest
```

Configuration (environment variables, set via `-e` on `docker run`): | Variable | Default | Meaning |
|---|---|---| | `PORT` | `8080` | Listen port inside the container | | `STORAGE_DIR` | `/data` | Storage root inside the container (mapped to `/localstorage/videos`) | | `MIN_FREE_GB` | `25` | Refuse uploads below this free space |

---

8. Security Notes

- Keep the service on the **private network only**; do not port-forward 8080 publicly.
- Add **authentication** (an API key/header check) and **TLS** if it must ever be reachable beyond the trusted network.
- Rotate any credentials that were shared to set this up.